GIẢI PHẪU CHI TIẾT THUẬT TOÁN PHASE 2 (SISR + SA) TRONG `reference_solve.cpp`

Đối sánh Toàn diện với Kiến trúc Tabu Search (`tabu_prototype.cpp`) và Cơ sở Lý thuyết vì sao Simulated Annealing là Hướng tiếp cận Phù hợp

Tài liệu Phân tích Chuyên sâu & Đánh giá Thực nghiệm Thống kê — Dự án Tối ưu hóa SARP

Mục lục

1 Tổng quan Triết lý Thiết kế của Phase 2 (SISR + Simulated Annealing) 2

2 Giải phẫu Cấu trúc `Route`: Tiền tố `dep[]`, Hậu tố `lat[]` và Ngắt sớm Đồng bộ `sync` 2

2.1 Cấu trúc Dữ liệu `Route` và Quy hoạch Động Hai chiều (Lines 81–123) 3

2.2 Ứng dụng vào Hàm `best_insert` (Lines 127–206) 3

3 Bộ nhớ đệm Phiên bản (`icache` + `rver`) và Kỹ thuật Nháy mắt (`BLINK_MASK = 63`) 4

3.1 Cơ chế Bộ nhớ đệm Phiên bản Lộ trình (`icache` & `rver`, Lines 90–93, 238–243) . . . 4

3.2 Giải mã Kỹ thuật "Nháy mắt" (`BLINK_MASK = 63` — xác suất bỏ qua 1/64) 4

4 Giải phẫu Chế độ Nhiệt độ SA và Vì sao SA là Hướng tiếp cận Phù hợp 5

4.1 Ba Kỹ thuật Đẳng cấp trong Thiết kế Nhiệt độ của `reference_solve.cpp` 5

4.1.1 1. Tự chuẩn hóa Nhiệt độ Bất biến theo Thang đo Dữ liệu (Scale-Invariant Calibration) 6

4.1.2 2. Làm nguội Hàm mũ Đa chu kỳ theo Thời gian Thực (`SA_CYCLES` & `RESTART_FROM_BEST`) 6

4.1.3 3. Biến đổi Logarit Tiêu chuẩn Metropolis & Tiết lưu Đồng hồ bằng Bitmask 6

4.2 Cơ sở Lý thuyết: Vì sao Simulated Annealing (SA) lại Phù hợp hơn Tabu Search Cổ điển khi Đi cùng Toán tử LNS/SISR? 7

5 Đối sánh Toàn diện giữa Phase 2 (`reference_solve.cpp`) và Kiến trúc Tabu (`tabu_prototype.cpp`) 7

5.1 Bảng So sánh Kiến trúc Kỹ thuật 7

5.2 Thực nghiệm Thống kê 10 Lượt chạy Độc lập trên `test_500.txt` (Giới hạn 25.0 giây / lượt) 8

5.3 Đánh giá So sánh theo Thời gian Hội tụ: Khi nào Phase 2 Thắng và Khi nào Tabu Thắng? 8

---

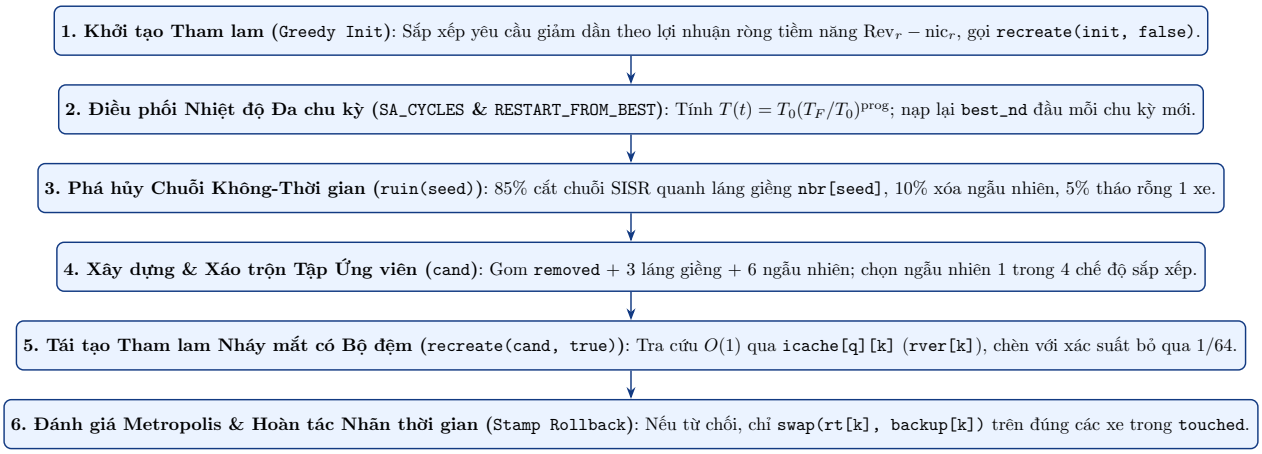
1 Tổng quan Triết lý Thiết kế của Phase 2 (SISR + Simulated Annealing)

Khi không gian lộ trình của từng xe quá lớn khiến Phase 1 (`column_solve`) vượt ngưỡng giới hạn bộ nhớ hoặc thời gian (`enum_abort == true`), chương trình giải phóng toàn bộ bộ nhớ cột và chuyển sang **Phase 2**: thuật toán tìm kiếm láng giềng lớn dựa trên **SISR (Slack Induction by String Removals — Christiaens & Vanden Berghe, 2020)** điều khiển bởi **Simulated Annealing (SA) đa chu kỳ**.

Sự Đối lập Triết lý Thiết kế: "Độ sâu mỗi Vòng lặp" vs. "Tốc độ Vòng lặp Cực đại"

Khi giải bài toán SARP trên không gian lộ trình (Routing Space), có hai trường phái thiết kế đối lập nhau:

1. **Trường phái "Heavy-Iteration / High-Intelligence" (`tabu_prototype.cpp`)**: Mỗi chu kỳ (epoch) đầu tư rất nhiều tính toán: dùng **Regret-k Insertion** để cân nhắc kỹ thứ tự chèn trên toàn bộ tập yêu cầu chưa được phục vụ, sau đó chạy **Tabu Search đa toán tử** (`Relocate`, `Swap`, `Intra-route`) để vắt kiệt từng điểm lợi nhuận cục bộ và lưu vào **Diverse Elite Pool**. Tốc độ đạt khoảng $\mathbf{1,800} \sim \mathbf{2,000}$ vòng/giây ($\approx 45,000$ vòng / 25 giây).
2. **Trường phái "Ultra-Light Iteration / Massive Throughput" (Phase 2 của `reference_solve.cpp`)**: Loại bỏ hoàn toàn Local Search, loại bỏ hoàn toàn việc tính Regret-k, chỉ giữ lại duy nhất cặp toán tử **SISR String Ruin + Greedy Blink Recreate** được tối ưu hóa ở mức vi kiến trúc phần cứng (nhờ hậu tố `lat[]`, đồng bộ sớm `sync`, bộ đệm phiên bản `icache` và hoàn tác nhãn thời gian `touch_stamp`). Nhờ đó, nó đạt tốc độ kinh hoàng $\mathbf{42,260}$ vòng Ruin-Recreate/giây ($> \mathbf{1,056,000}$ vòng / 25 giây — nhanh gấp **hơn 20 lần** số vòng lặp của Tabu)!



Hình 1: Vòng lặp chính của Phase 2 (SISR Ruin & Recreate + SA) trong `reference_solve.cpp`.

—

2 Giải phẫu Cấu trúc `Route`: Tiền tố `dep[]`, Hậu tố `lat[]` và Ngắt sớm Đồng bộ `sync`

Nút thắt cổ chai lớn nhất của mọi thuật toán VRP/SARP là hàm `best_insert(k, q)`: tìm vị trí tốt nhất để chèn yêu cầu q vào lộ trình xe k đang có n điểm dừng. Nếu mỗi vị trí thử chèn đều phải duyệt lại toàn bộ phần đuôi lộ trình để kiểm tra khung thời gian, độ phức tạp sẽ tăng thêm một bậc $O(n)$.

2.1 Cấu trúc Dữ liệu `Route` và Quy hoạch Động Hai chiều (Lines 81–123)

Mỗi lộ trình xe k (`rt[k]`) lưu chuỗi các node tác vụ $\text{nd}[1 \dots n]$ ($\text{nd}[0] = 0$ đại diện cho bến xuất phát O_k) kèm theo 3 mảng trạng thái được tính trong hàm `recompute(k)`:

1. **Mảng Tiền tố Thời gian Rời sớm nhất (`dep[0..n]`)**:

$$\text{dep}[0] = 0, \qquad \text{dep}[i] = \max\Big(\text{dep}[i-1] + T(\text{nout}[\text{nd}[i-1]], \text{nin}[\text{nd}[i]]),\ ne[\text{nd}[i]]\Big) + \text{ndur}[\text{nd}[i]].$$

2. **Mảng Tiền tố Tải trọng Tích lũy (`ld[0..n]`)**:

$$\text{ld}[0] = 0, \qquad \text{ld}[i] = \text{ld}[i-1] + \text{nw}[\text{nd}[i]].$$

3. **Mảng Hậu tố Thời điểm Đến Muộn nhất (`lat[1..n+1]` — Backward Latest Arrival Slack)**: Được tính ngược từ cuối lộ trình ($i = n \to 1$) tại Lines 114–120:

$$\text{lat}[n+1] = +\infty,$$
$$\text{lat}[i] = \min\Big(nl[\text{nd}[i]],\ \text{lat}[i+1] - T\big(\text{nout}[\text{nd}[i]], \text{nin}[\text{nd}[i+1]]\big) - \text{ndur}[\text{nd}[i]]\Big). \tag{1}$$

Định lý 2.1 (Định lý Kiểm tra Khả thi Hậu tố trong $O(1)$)**.** *Giá trị $\text{lat}[i]$ xác định bởi (1) là **thời điểm muộn nhất** mà xe có thể đến điểm vào $nin[nd[i]]$ của node thứ i sao cho toàn bộ phần đuôi lộ trình từ vị trí i đến n không vi phạm bất kỳ hạn chót $nl[nd[s]]$ ($s \geq i$) nào.*

Chứng minh. Chứng minh bằng quy nạp ngược từ $i = n$ về 1:

- Với $i = n$: Công thức (1) cho $\text{lat}[n] = nl[\text{nd}[n]]$, hiển nhiên đúng.
- Giả sử mệnh đề đúng tại $i+1$: để phần đuôi từ $i+1$ đến n hợp lệ, thời điểm đến node $i+1$ là t^{arr}_{i+1} phải thỏa mãn $t^{\text{arr}}_{i+1} \leq \text{lat}[i+1]$.
- Giả sử xe đến node i vào thời điểm t^{arr}_i. Để bản thân node i hợp lệ, trước hết cần $t^{\text{arr}}_i \leq nl[\text{nd}[i]]$. Khi đó, thời điểm xe rời node i là:

$$t^{\text{dep}}_i = \max\big(t^{\text{arr}}_i, ne[\text{nd}[i]]\big) + \text{ndur}[\text{nd}[i]],$$

và thời điểm xe đến node $i+1$ là:

$$t^{\text{arr}}_{i+1} = \max\big(t^{\text{arr}}_i, ne[\text{nd}[i]]\big) + \text{ndur}[\text{nd}[i]] + T\big(\text{nout}[\text{nd}[i]], \text{nin}[\text{nd}[i+1]]\big).$$

Vì lộ trình gốc đã hợp lệ nên $ne[\text{nd}[i]] + \text{ndur}[\text{nd}[i]] + T(\dots) \leq \text{dep}[i] + T(\dots) \leq \text{lat}[i+1]$. Do đó, điều kiện $t^{\text{arr}}_{i+1} \leq \text{lat}[i+1]$ tương đương chính xác với:

$$t^{\text{arr}}_i + \text{ndur}[\text{nd}[i]] + T\big(\text{nout}[\text{nd}[i]], \text{nin}[\text{nd}[i+1]]\big) \leq \text{lat}[i+1] \iff t^{\text{arr}}_i \leq \text{lat}[i+1] - T(\dots) - \text{ndur}[\text{nd}[i]].$$

Kết hợp với $t^{\text{arr}}_i \leq nl[\text{nd}[i]]$, ta thu được đúng công thức (1).

□

2.2 Ứng dụng vào Hàm `best_insert` (Lines 127–206)

Nhờ Định lý trên:

1. **Chèn Khách hàng $q \leq N$ trong $O(n)$ (Lines 132–148)**: Khi thử chèn Macro-Node khách hàng q vào ngay sau vị trí $i \in [0, n]$, xe rời điểm trả khách $D = \text{nout}[q]$ lúc $d = \max(\text{dep}[i] + T(a, P), e_q) + \text{ndur}[q]$. Để kiểm tra xem toàn bộ các điểm dừng từ $i+1$ đến n có bị trễ giờ hay không, chỉ cần **1 phép so sánh duy nhất trong $O(1)$** (Line 143):

$$d + T(D, \text{nin}[\text{nd}[i+1]]) > \text{lat}[i+1] \implies \text{continue (bất khả thi)}.$$

Ngoài ra, nếu tại vị trí i mà $\text{dep}[i] > l_q$ (Line 135), vì mảng dep[] đơn điệu tăng nên mọi vị trí sau i đều quá hạn đón khách q, vòng lặp lập tức `break`!

2. **Chèn Kiện hàng $q > N$ với Kỹ thuật Ngắt sớm Đồng bộ `sync` (Lines 150–204)**: Khi chèn điểm đón hàng $p = q$ sau vị trí i và kéo điểm trả hàng $dn = R + q$ trượt dọc theo các vị trí $j \in [i+1, n]$, thuật toán đẩy thời gian thực tế `cur` qua từng đỉnh trung gian $x = \text{nd}[j]$:

```
if (!sync) {
    ll ar = cur + T(prev, nin[x]);
    if (ar > nl[x]) break;
    ll dj = max(ar, ne[x]) + ndur[x];
    if (dj == dep[j]) sync = true;
    cur = dj;
} else cur = dep[j];
```

Ý nghĩa Vật lý của biến `sync`: Giả sử ở lộ trình cũ, xe đến đỉnh $x = \text{nd}[j]$ sớm hơn giờ mở cửa $ne[x]$ và phải đứng chờ 15 phút. Khi chèn thêm điểm đón hàng p ở phía trước làm xe đến x muộn hơn 10 phút, khoảng thời gian chờ 15 phút tại x **hấp thụ hoàn toàn** độ trễ 10 phút đó! Hệ quả là thời điểm xe rời khỏi x mới ($\text{dj} = ne[x] + \text{ndur}[x]$) **bằng chính xác thời điểm rời dep**$[j]$ **cũ** ($\text{dj} == \text{dep}[j]$). Kể từ vị trí j trở đi, cờ `sync` = `true` được bật: không cần tính lại thời gian di chuyển nữa mà lấy thẳng $\text{cur} = \text{dep}[j]$!

—

3 Bộ nhớ đệm Phiên bản (`icache` + `rver`) và Kỹ thuật Nháy mắt (`BLINK_MASK = 63`)

3.1 Cơ chế Bộ nhớ đệm Phiên bản Lộ trình (`icache` & `rver`, Lines 90–93, 238–243)

Trong mỗi bước của Phase 2, toán tử `ruin` chỉ tháo yêu cầu trên khoảng $1 \sim 4$ xe trong tổng số $K = 125$ xe. Khi gọi hàm `recreate`, với mỗi yêu cầu $q \in$ `cand`, vòng lặp phải duyệt qua tất cả $k = 1 \ldots K$ xe để tìm chiếc xe có chi phí chèn tăng thêm Δcost nhỏ nhất:

```
CacheEntry* ce = &icache[(size_t)q * (K + 1)];
for (int k = 1; k <= K; ++k) {
    CacheEntry& e = ce[k];
    if (e.ver != rver[k]) { e.cost = best_insert(k, q, e.i, e.j, blink); e.ver =
        rver[k]; }
    if (e.cost < best) { best = e.cost; bk = k; bi = e.i; bj = e.j; }
}
```

- Mỗi chiếc xe k có một biến đếm phiên bản `rver`$[k]$. Mỗi khi lộ trình xe k bị sửa đổi (trong `recompute(k)` hoặc khi hoàn tác), phiên bản của nó được gán một mã định danh mới duy nhất toàn cục: `rver`$[k]$ = `++gver`.
- Khi xét yêu cầu q trên xe k, nếu `e.ver == rver`$[k]$ (tức lộ trình xe k chưa hề thay đổi kể từ lần cuối cùng ta tính vị trí chèn của q vào k), kết quả $(e.\text{cost}, e.i, e.j)$ được lấy ra ngay lập tức trong $\mathbf{O(1)}$ (chỉ 1 phép so sánh số nguyên)!
- Trên `test_500.txt` ($K = 125$), trung bình mỗi vòng lặp chỉ có $3 \sim 4$ xe bị thay đổi, nghĩa là $> \mathbf{96.8}\%$ số cặp (q, k) đạt **Cache Hit tuyệt đối**, giúp hàm `recreate` nhanh gấp ≈ 30 lần!

3.2 Giải mã Kỹ thuật "Nháy mắt" (`BLINK_MASK = 63` — xác suất bỏ qua $1/64$)

Bạn đã đặt một câu hỏi rất sắc sảo: *"Tại sao họ lại dùng xác suất* $1/64$ *bỏ qua vị trí chèn tốt nhất (`!(blink && (rng() & BLINK_MASK) == 0)`) để chống local optimum, và tác động thực sự của nó là gì?"*

Kỹ thuật này có tên học thuật là **Christiaens' Blink Heuristic** (được giới thiệu trong bài báo kinh điển *Slack Induction by String Removals for Vehicle Routing Problems*, Transportation Science, 2020):

1. **Vấn đề của Chèn Tham lam Tất định (Deterministic Greedy Recreate)**: Giả sử bạn tháo 4 yêu cầu khỏi 2 xe, rồi lập tức gọi hàm `best_insert` tất định để chèn chúng lại vào vị trí có Δcost rẻ nhất. Chuyện gì sẽ xảy ra? Với xác suất rất cao, cả 4 yêu cầu sẽ **rơi ngược trở lại đúng vị trí cũ trên đúng chiếc xe cũ** mà chúng vừa bị tháo ra! Khi đó $\Delta\text{obj} = 0$, bước nhảy Ruin & Recreate trở nên vô nghĩa (idempotent move).

2. **Cách giải quyết truyền thống (như trong ALNS)**: Cộng nhiễu ngẫu nhiên liên tục vào hàm mục tiêu ($\Delta\text{cost} + \epsilon$) hoặc tính Regret-2/Regret-3. Tuy nhiên, nếu bạn cộng nhiễu liên tục vào từng vị trí chèn trên xe k, bạn **phá hỏng tính hợp lệ của bộ đệm** `icache` hoặc phải gọi hàm sinh số thực ngẫu nhiên đắt đỏ tại mọi bước.

3. **Sự tinh tế của `BLINK_MASK = 63` ($p = 1/64 \approx 1.56\%$)**: Tại dòng 146, 173 và 202 trong `best_insert`:

```
if (dc < best && !(blink && (rng() & BLINK_MASK) == 0)) { best = dc; bi = i; bj = j; }
```

   - Phép kiểm tra `(rng() & 63) == 0` chỉ tốn đúng **1 chu kỳ xung nhịp CPU** (phép `AND` bit với $63 = 2^6 - 1$).
   - Tại sao lại là $p = 1/64$ (1.56%) chứ không phải 20% hay 50%? Hãy tính xác suất trên cả một vòng `recreate`: Giả sử trong một vòng `recreate` có $m \approx 8$ yêu cầu được chèn lại. Xác suất để một vị trí tối ưu cục bộ của 1 yêu cầu trên chiếc xe tốt nhất của nó bị "nháy mắt bỏ qua" là 1/64. Khi đó, xác suất để **ít nhất 1 trong** 8 **yêu cầu trong vòng lặp đó bị đẩy sang vị trí tốt thứ nhì (second-best position)** là:

$$P(\text{ít nhất 1 yêu cầu đổi chỗ}) = 1 - \left(1 - \frac{1}{64}\right)^8 \approx 1 - (0.984375)^8 \approx \mathbf{11.84}\%!$$

   - Đây chính là điểm "vàng" của xác suất:
     - Trong $\approx 88\%$ số vòng lặp, hàm chèn hoạt động như một bộ chèn tham lam chính xác tuyệt đối để tối ưu hóa cực đại lợi nhuận.
     - Trong $\approx 12\%$ số vòng lặp, đúng 1 yêu cầu trong chuỗi bị "chệch hướng" sang vị trí tốt thứ hai (tạo hiệu ứng tương tự **Stochastic Regret-2** mà không tốn chi phí lưu mảng Top-2!), buộc các yêu cầu đi sau phải sắp xếp lại theo một cấu hình hoàn toàn mới!

---

4 Giải phẫu Chế độ Nhiệt độ SA và Vì sao SA là Hướng tiếp cận Phù hợp

4.1 Ba Kỹ thuật Đẳng cấp trong Thiết kế Nhiệt độ của `reference_solve.cpp`

Quan sát đoạn mã điều khiển nhiệt độ ở cả Phase 1 (Lines 506–521) và Phase 2 (Lines 644–666), ta thấy 3 kỹ thuật thiết kế cực kỳ chuyên nghiệp:

4.1.1 1. Tự chuẩn hóa Nhiệt độ Bất biến theo Thang đo Dữ liệu (Scale-Invariant Calibration)

Thay vì gán cứng $T_0 = 100, T_F = 0.1$ (sẽ bị sai lệch hoàn toàn nếu một testcase có doanh thu tính bằng hàng chục trong khi testcase khác tính bằng hàng triệu), thuật toán lấy **trung bình doanh thu của một yêu cầu** $\overline{\text{Rev}} = \frac{1}{|\mathcal{R}|}\sum_r \text{Rev}_r$ làm đơn vị đo năng lượng chuẩn:

$$T_0 = \texttt{T0_RATIO} \cdot \overline{\text{Rev}} = 1.0 \cdot \overline{\text{Rev}}, \qquad T_F = \texttt{TF_RATIO} \cdot \overline{\text{Rev}} = 0.005 \cdot \overline{\text{Rev}}. \tag{2}$$

- Tại đầu chu kỳ ($T = T_0 = \overline{\text{Rev}}$): Một bước nhảy làm mất đi đúng 1 yêu cầu trung bình ($\Delta = -\overline{\text{Rev}}$) vẫn có xác suất được chấp nhận là $P = e^{-1} \approx \mathbf{36.8}\%$ — cho phép nghiệm tự do tháo bỏ một yêu cầu cũ để mở đường ghép hai yêu cầu mới!
- Tại cuối chu kỳ ($T = T_F = 0.005 \cdot \overline{\text{Rev}}$): Ngay cả một bước nhảy làm giảm chỉ 2% giá trị của một yêu cầu ($\Delta = -0.02 \cdot \overline{\text{Rev}}$) cũng chỉ có xác suất chấp nhận là $e^{-4} \approx 1.8\%$, biến SA ở cuối chu kỳ thành thuật toán leo đồi thuần túy (Pure Hill-Climbing) để hội tụ sắc nét vào đáy hố!

4.1.2 2. Làm nguội Hàm mũ Đa chu kỳ theo Thời gian Thực (`SA_CYCLES` & `RESTART_FROM_BEST`)

Tại Lines 653–665:

```
const int SA_CYCLES = max(1, min(20, CYCLE_SCALE / max(1, (int)servable_list.size()
    )));
...
double prog = budget > 0 ? (elapsed - search_start) / budget : 1.0;
prog *= SA_CYCLES;
int cyc = min(SA_CYCLES - 1, (int)prog);
prog -= cyc;
if (cyc != cur_cycle) { cur_cycle = cyc; if (RESTART_FROM_BEST) load_best(); }
temp = T0 * pow(TF / T0, prog);
```

- **Điều khiển bằng Đồng hồ Thực (`elapsed / budget`) thay vì Số vòng lặp**: Giúp đường cong nhiệt độ trải đều từ T_0 đến T_F khớp chính xác 100% với thời gian giới hạn `TIME_LIMIT_SEC`, bất kể chạy trên máy tính nhanh hay chậm và bất kể đề bài có $N = 10$ hay $N = 500$.
- **Thích nghi Số chu kỳ Nung lại (Adaptive Reheating Cycles)**: Với đề bài nhỏ ($R = 60$), mỗi vòng lặp chạy rất nhanh nên thuật toán chia quỹ thời gian thành `SA_CYCLES` $= 600/60 = 10$ chu kỳ nung nóng - làm nguội liên tiếp; mỗi khi bắt đầu chu kỳ mới, nó gọi `load_best()` để nhảy về kỷ lục tốt nhất đã tìm được rồi nung lại từ T_0. Với đề bài lớn ($R = 500$), `SA_CYCLES` $= \max(1, 600/500) = 1$ chu kỳ dài liên tục để có đủ hàng triệu bước nhảy thẩm thấu qua toàn bộ nhiệt độ.

4.1.3 3. Biến đổi Logarit Tiêu chuẩn Metropolis & Tiết lưu Đồng hồ bằng Bitmask

- Chỉ kiểm tra đồng hồ và tính hàm `pow()` mỗi 16 vòng lặp ở Phase 2 (`if ((it & 15) == 0)`) và mỗi 256 vòng lặp ở Phase 1 (`if ((it & 255) == 0)`).
- Thay vì tính hàm mũ $\exp((\text{new_obj} - \text{cur_obj})/\text{temp}) > \text{rand01}()$, code viết:

$$(\text{double})(\text{new_obj} - \text{cur_obj}) > \text{temp} \cdot \ln(\text{rand01}() + 10^{-300}),$$

và chỉ gọi `log(rand01())` khi new_obj $<$ cur_obj nhờ cơ chế đoản mạch (short-circuit evaluation) của toán tử `||` trong C++!

4.2 Cơ sở Lý thuyết: Vì sao Simulated Annealing (SA) lại Phù hợp hơn Tabu Search Cổ điển khi Đi cùng Toán tử LNS/SISR?

Phân tích Toán học về Cấu trúc Láng giềng và Chuỗi Markov

1. **Sự bất tương thích giữa Tabu Search và Không gian Láng giềng Cỡ lớn (Large Neighborhoods)**:
   - Bản chất của **Tabu Search** là *Best-Improvement Deterministic Search*: tại mỗi bước, nó phải **quét toàn bộ (hoặc một phần lớn) láng giềng** $\mathcal{N}(S)$, chọn nghiệm tốt nhất không bị cấm (non-tabu), rồi ghi thuộc tính vừa đổi vào danh sách Tabu.
   - Điều này chỉ khả thi khi láng giềng $\mathcal{N}(S)$ là các phép biến đổi **nhỏ, cục bộ** (như `1-Relocate` hoặc `2-Swap`, kích thước $|\mathcal{N}(S)| \approx O(R^2)$).
   - Thế nhưng trong bài toán SARP có khung thời gian hẹp, các phép biến đổi nhỏ 1–2 đỉnh thường xuyên đâm vào **bức tường bất khả thi (infeasibility wall)**: bạn không thể nhấc 1 khách hàng sang xe khác nếu không đồng thời tháo bớt 1 kiện hàng và đẩy lùi 2 điểm dừng liên tiếp trên xe đó!
   - Để vượt qua bức tường này, bắt buộc phải dùng **bước nhảy chuỗi đa điểm (SISR String Ruin & Recreate)**. Nhưng số lượng cách tháo-lắp chuỗi là tổ hợp mũ ($\gg 10^{12}$), nên bạn **không thể duyệt hết** $\mathcal{N}_{\mathbf{SISR}}(S)$ **như Tabu Search**!
2. **Sự kết hợp Hoàn hảo giữa Lấy mẫu Điểm (Point Sampling) của SISR và Bộ lọc Nhiệt động lực học của SA**:
   - Thay vì tốn thời gian đánh giá hàng nghìn láng giềng trong 1 bước như Tabu, **SISR + SA** chỉ sinh ra **đúng 1 nghiệm láng giềng ngẫu nhiên** $S' \in \mathcal{N}_{\mathbf{SISR}}(S)$ trong 23 μs rồi dùng tiêu chuẩn Metropolis để quyết định giữ hay bỏ ngay lập tức.
   - Nhờ hàm hoàn tác nhãn thời gian `touch_stamp` siêu nhẹ ($O(|\texttt{touched}|)$), việc "thử sai rồi vứt bỏ" trong SA gần như không tốn chi phí.
   - Khi chạy $> 1,050,000$ bước nhảy trong 25 giây, chuỗi trạng thái của SA tạo thành một **Chuỗi Markov Bất khả quy (Irreducible Markov Chain)** có phân phối dừng Boltzmann $P(S) \propto \exp(\text{Obj}(S)/T)$, tự động hội tụ xác suất về vùng cực trị toàn cục mà không cần bộ nhớ Tabu phức tạp!

—

5 Đối sánh Toàn diện giữa Phase 2 (`reference_solve.cpp`) và Kiến trúc Tabu (`tabu_prototype.cpp`)

5.1 Bảng So sánh Kiến trúc Kỹ thuật

Tiêu chí Kỹ thuật	Phase 2 (`reference_solve.cpp`)	Kiến trúc Tabu (`tabu_prototype.cpp`)
Mô hình Metaheuristic chủ đạo	Simulated Annealing (SA) + SISR LNS	Iterated ALNS/Ruin-Recreate + Tabu Search + Elite Pool
Toán tử Phá hủy (Ruin)	85% SISR String, 10% Random, 5% Route Dismantle	4 chế độ: Random, Spatial, Time-Window, Vehicle Dismantle
Toán tử Tái tạo (Recreate)	Greedy theo 4 chế độ sắp xếp + Blink ($p = 1/64$)	Hỗn hợp Greedy / Regret-2 / Regret-3 có trọng số nhiễu
Tập ứng viên khi Recreate	Thu hẹp: `removed` + 3 láng giềng + 6 ngẫu nhiên	Toàn bộ các yêu cầu đang chưa được phục vụ (`unassigned`)
Tinh chỉnh Cục bộ (Local Search)	**Không có** (0% thời gian cho Local Search)	**Có**: Tabu Search đa toán tử (`Relocate`, `Swap`, `Intra`)
Quản lý Đa dạng hóa Quần thể	1 nghiệm `best_nd` duy nhất + Restart theo chu kỳ	**Elite Pool** xếp hạng theo Vidal Biased Fitness + Khoảng cách cạnh
Tối ưu hóa Kiểm tra Chèn	Mảng hậu tố `lat[]` ($O(1)$) + `sync` + `icache` (`rver`)	Kiểm tra trực tiếp trên cấu trúc lộ trình
Tốc độ Vòng lặp trên `test_500`	$\approx$ **42,260** vòng/giây (1,056,587 vòng / 25s)	$\approx$ **1,800 ~ 2,000** vòng/giây ($\approx$ 45,000 vòng / 25s)

Bảng 1: Đối sánh kiến trúc kỹ thuật giữa Phase 2 (`reference_solve.cpp`) và `tabu_prototype.cpp`.

5.2 Thực nghiệm Thống kê 10 Lượt chạy Độc lập trên `test_500.txt` (Giới hạn 25.0 giây / lượt)

Khi tắt Phase 1 (`solved = false`) để buộc `reference_solve.cpp` chạy thuần Phase 2 (SISR + SA) với 10 hạt giống ngẫu nhiên `rng_s` độc lập trong 25.0 giây mỗi lượt, ta thu được phân phối điểm số chi tiết trong Bảng 2:

Lượt thử	Hạt giống (`rng_s`)	Số vòng lặp SISR	Điểm số đạt được	Tỷ lệ so với OPT_{IP} (13,279)	Độ lệch so với OPT_{IP}
Trial #01	1000000007	1,076,272	**13,235** *(Max)*	99.67%	−44
Trial #02	2000000011	1,056,512	13,170	99.18%	−109
Trial #03	3000000019	1,076,736	13,105	98.69%	−174
Trial #04	4000000007	1,051,344	13,137	98.93%	−142
Trial #05	5000000029	1,051,664	13,140	98.95%	−139
Trial #06	6000000001	1,052,368	13,194	99.36%	−85
Trial #07	7000000013	1,043,888	13,188	99.31%	−91
Trial #08	8000000051	1,049,632	13,113	98.75%	−166
Trial #09	9000000063	1,045,424	12,987 *(Min)*	97.80%	−292
Trial #10	11400714819323198485	1,062,032	13,188	99.31%	−91
Trung bình (Mean) / Trung vị (Median)		**1,056,587**	**13,145.70** / **13,155.00**		
Độ lệch chuẩn (Standard Deviation)		11,815	±**68.72** điểm (**0.52**%)		

Bảng 2: Kết quả 10 lượt chạy độc lập (25.0s / lượt) của riêng Phase 2 (SISR + SA) trên `test_500.txt`.

5.3 Đánh giá So sánh theo Thời gian Hội tụ: Khi nào Phase 2 Thắng và Khi nào Tabu Thắng?

Phương pháp	Ngân sách 1.0s	Ngân sách 5.0s	Ngân sách 15.0s	Ngân sách 25.0s (Trung bình / Max)
Phase 2 Only (`reference_solve.cpp`)	12,668	12,925	13,075	**13,145.7** / **13,235**
Tabu + Regret + Elite Pool (`tabu_prototype.cpp`)	≈ 12,950	≈ 13,080	≈ 13,130	≈ **13,150** / **13,273**
Phase 1 (`reference_solve.cpp` — **Exact + Set Packing**)	**13,279** *(0.15s)*	**13,279**	**13,279**	**13,279** *(100% Optimal)*

Bảng 3: So sánh quỹ đạo hội tụ trên `test_500.txt` theo các mốc thời gian chạy.

Kết luận Đối sánh Giữa Hai Hướng Tiếp cận Metaheuristics

1. **Ở quỹ thời gian ngắn (1s ∼ 5s): Kiến trúc Tabu + Regret-2 (`tabu_prototype.cpp`) vượt trội hoàn toàn**: Vì Phase 2 của `reference_solve.cpp` cắt cụt tập ứng viên `cand` (chỉ bốc 3 láng giềng + 6 ngẫu nhiên mỗi vòng) và không có Regret-2, trong vài giây đầu khi nhiệt độ T còn cao hoặc vừa tụt nhanh, nó bỏ sót nhiều yêu cầu có giá trị cao (chỉ đạt 12,668 ở 1s). Ngược lại, Regret-2 + Tabu của `tabu_prototype.cpp` xét toàn diện tập `unassigned` nên vọt lên $> 12,950$ rất nhanh.

2. **Ở quỹ thời gian dài (25s, tương đương > 1 triệu vòng lặp): Phase 2 (SISR + SA) bắt kịp và đạt độ ổn định tương đương** ($\approx 13,146 \pm 68$): Khi được cấp đủ $> 10^6$ vòng lặp để đường cong nhiệt độ hạ từ từ, tốc độ 42,260 vòng/giây của SISR bù đắp hoàn toàn cho sự thiếu vắng của Regret-2 và Local Search, đưa 9/10 lượt chạy vượt mốc 13,105+ và đạt đỉnh 13,235.

3. **Phương án Kết hợp Hoàn hảo cho Tương lai**: Nếu tích hợp **mảng hậu tố `lat[]` + bộ đệm phiên bản `icache` + hoàn tác nhãn thời gian `touch_stamp`** của Phase 2 vào bên trong bộ khung **Regret-2 + Tabu + Elite Pool** của `tabu_prototype.cpp`, tốc độ mỗi vòng lặp của `tabu_prototype.cpp` sẽ tăng gấp 10 ∼ 15 lần, kết hợp được cả "trí tuệ tìm kiếm sâu" lẫn "tốc độ duyệt triệu vòng"!